TECHNICAL REVIEW · DEEP-DIVE AUDIT

AnyDisasterMapping

Architecture, gap audit, and extension assessment of the config-driven PyTorch benchmark for satellite-based disaster mapping (seg / cd / scd), including runtime verification of the 9-domain extension.

7.5/10	3.5/10	~8/10
AS RESEARCH FRAMEWORK	AS DEMO TOOLKIT (BEFORE)	AS DEMO TOOLKIT (AFTER)

Scope: full repository audit · static analysis of all core modules · dependency audit · runtime verification (install, CLI, 12-run smoke matrix)
Audience: research team (benchmark maintainers & contributors)
Basis: AnyDisasterMapping-master + 2026 extension (9 domains, Colab demo, docs)

REVIEW — AnyDisasterMapping: Architecture, Gaps, and Extension Assessment

Reviewer scope: full repository audit (`AnyDisasterMapping-master`), static analysis of every core module, dependency audit, and runtime verification (install, CLI, 1-epoch dummy-data training + testing for segmentation and change-detection task types on CPU). **Audience:** research team (benchmark maintainers and contributors). **Verdicts:** as a *research framework* **7.5/10** · as a *demo-ready toolkit* **3.5/10** (before this extension) — see §8.

1. Executive Summary

AnyDisasterMapping is the official companion repository of the review paper *"Earth Observation for Disaster Mapping: Benchmarks, Methods, Challenges and Future Perspectives"* (Chen et al., SSRN 6725082, RIKEN AIP / UTokyo / UN Satellite Centre et al.). It unifies 13 satellite disaster-mapping datasets across 4 domains (flood, infrastructure damage, landslide, wildfire) with 31 registered model factories behind a single config-driven trainer supporting three task contracts: segmentation (`seg`), change detection (`cd`), and semantic change detection (`scd`).

The engineering quality of the core is **high**. The trainer (809 lines) is a genuinely unified runtime with three loss branches, lazy scheduler construction, auto-resume, and sliding-window inference; the task handlers cleanly separate tuple contracts; the registry uses lazy imports so heavy model dependencies are only paid when a model is actually instantiated. Runtime verification for this review confirmed the full train→test loop works end-to-end on synthetic data for both `seg` and `cd` (11/12 smoke runs pass; the 12th requires a GPU triton kernel).

The gaps are concentrated in **productization**, not science: no trained checkpoints ship with the repo, there is no demo or single-image inference path, no test suite, a placeholder LICENSE, and one task (`scd`) is fully wired in code but was never exposed through a single config. All five gap classes are closed or mitigated by the accompanying extension (9 new domains, `demo_colab.ipynb` , smoke tests, Apache-2.0 text, 20 configs including the repo's first `scd` config).

2. Method

Step	What was done
Inventory	608 YAML configs; 34 registry entries (31 factories + 3 aliases); 15 dataset entries (13 datasets + 2 aliases); 24 dataset modules after extension

Static review	Read in full: `train.py`, `test.py`, `src/core/{trainer,config,registry,augmentation}.py`, `src/tasks/{base,cd,scd}.py`, `src/datasets/{flood_cau,infra_xbd,second}.py`, `docs/*.md`, `pyproject.toml`, `README.md`, `LICENSE`; structural review of all model wrappers' `__init__` signatures
Runtime	`pip install -e .` (CPU torch 2.7.1), `train.py --help`, `test.py --help`, registry round-trip for all 9 new adapters, config parsing, 1-epoch dummy train+test across 12 task/domain combinations
Gap audit	Checklist from the extension brief: checkpoints, Colab/demo, single-image inference, LICENSE validity, tests, dependency pinning

3. Architecture Review

3.1 Runtime flow

The entry points are deliberately thin. `train.py` (28 lines) parses `--config/--override/--resume`, builds a `Config`, and hands off to `Trainer`. `test.py` (180 lines) adds experiment-directory conventions: it loads `config.yaml` saved next to the checkpoint, supports multiple `--test_splits` with confusion-matrix aggregation across splits, and a freshness check (`_is_fresh`) that skips re-evaluation when `test_metrics.json` is newer than the checkpoint. This freshness logic is a thoughtful touch for long benchmark sweeps, though the multi-split path (test.py:73–157) re-evaluates *all* splits if any is stale because confusion matrices cannot be restored from scalar JSONs — correct, but worth documenting for users running expensive test sets.

`Trainer` owns the runtime: model build (`_build_model`, trainer.py:167), optimizer with optional encoder/decoder differential learning rates (`_infer_encoder_param_prefixes`, trainer.py:62 — auto-detects `encoder` / `backbone` / `image_encoder` / `spat_encoder` / `spec_encoder` attributes or honors an explicit `encoder_param_prefixes`), lazily-built scheduler (`_build_scheduler` is deferred to `train()` because `WarmUpPolyLR` needs `len(train_loader)`, trainer.py:224–259), early stopping, checkpointing (`best.pth` = weights only; `latest.pth` = full resumable state including optimizer/scheduler/early-stopping), and evaluation with optional sliding-window inference for images larger than the training crop.

3.2 Registries and lazy imports

`src/core/registry.py` implements two dicts of `lazy_import` factories. Strengths: negligible import cost for the 31-model zoo; alias entries (`deeplabv3p`, `satmaedpt`, `sam2`) forgive naming drift; metadata tables `DUAL_INPUT_MODELS` (named dual-input forwards, e.g. `bit: (x1, x2)`) and `DEEP_SUPERVISION_MODELS` (per-scale loss weights for DSIFN/UNet++) keep model quirks declarative rather than hard-coded in the trainer. HRNet's yacs-config factory is a special case handled cleanly.

Weaknesses: registry names are strings with no validation surface — a typo in `model.name` produces a `KeyError` at trainer construction rather than a helpful "closest match" message. Registry count (31 model factories) matches the paper's claim, so the zoo number is honest.

3.3 Task contracts

The three task handlers define the repository's most important interface — the dataset tuple contract:

Task	Dataset returns	Label semantics	Metrics
`seg`	`(image (C,H,W) f32, label (H,W) i64, id)`	`255 = ignore`	OA, mIoU, per-class IoU/F1
`cd`	`(pre, post, label, id)` or xBD-style `(pre, post, loc, clf, id)`	binary `loc = label > 0` ; `255` propagates	OA, mIoU, `loc_f1` , `clf_f1`
`scd`	`(pre, post, cd_label, t1_label, t2_label, id)`	semantic `0` masked to ignore during loss; models return `(out_cd, out_t1, out_t2)`	IoU_mean, SeK, Fscd

Augmentation synchronization is resolved by the *task handler* (`augmentation_targets()`), not the dataset: `cd` adds `image_post` , `scd` additionally adds `mask_t1` / `mask_cd` as albumentations `additional_targets` . This is the right place for it — datasets stay dumb, and `build_transforms` (augmentation.py:172) consults `get_task_handler(task).augmentation_targets()` so configs never repeat that wiring.

3.4 Trainer loss branches

`_train_step` (trainer.py:447) dispatches three branches: (1) Mask2Former-style built-in loss via `model_has_builtin_loss` ; (2) deep supervision for registered models with weighted per-scale CE(+Lovász); (3) task-driven loss — dual-head tuples get `criterion_loc(loc) + criterion_clf(clf)` , and `TaskSCD.compute_loss` reproduces ChangeMamba's SCD objective exactly (CD loss + 0.5·(T1 + T2 + 0.5·MSE consistency on ignored regions), scd.py:27–49). An `ignore-all-255` batch shortcut (trainer.py:456) avoids wasted backward passes on fully-invalid tiles — good for damage datasets where whole chips can be nodata.

The `_build_loss` closure (trainer.py:302) supports CE+dice and CE+Lovász with mutually-exclusive flags (dice wins), per-head weights, and a shared `ignore_index` . One subtlety: dice/lovasz are computed from the *resolved* logits, so deep-supervision models weight CE per scale but the auxiliary terms only apply to the primary output — acceptable, but undocumented.

3.5 Config system

`Config` is a flat-YAML wrapper with dot-access, `get("a.b", default)` , CLI override casting (`true/false/int/float/none`), and `save_yaml` . The trainer's dataset-build path introspects each adapter's `__init__` signature (by unwrapping the lazy-import closure cells, trainer.py:375–407) to inject `task=` or `input_cfg=` **only when accepted** — clever, though it reaches into closure internals (`factory.__closure__`), which is fragile against any refactor of `lazy_import` .

4. Model Zoo & Dataset Coverage

Models (31 factories): classics (UNet, UNet++, DeepLabV3+, FCN8s, HRNet w18/w48), RS-specific (FarSeg/FarSegPP, UNetFormer, SkySenseUPerNet), transformer-era (SegFormer b0–b5 wrappers, SwinUperNet, ConvNeXtUPerNet, Mask2Former), foundation-model adapters (DINOv2/v3 DPT + LoRA variant, SAM/SAM2 DPT+FPN, CLIP-DPT, SatMAE-DPT, SpectralGPT, HyperSigma), and CD

specialists (BIT, SiamCRNN, ChangeOS, DSIFN, ChangeMamba, RS3Mamba, DamageFormer, ChangeMamba-SCD). Coverage is current through 2025 architectures (DINOv3, SAM 2.1, Mamba) — a genuine differentiator versus mmseg-based disaster benchmarks.

Datasets (13): flood (CAU, KuroSiwo, UrbanSARFloods), infrastructure damage (xBDDataset, BRIGHT, RescueNet), landslide (GVLM, Landslide4Sense, HRGLDD), wildfire (S2WCD, FLOGA, SatelliteBurnedArea, FireSpread) + SECOND for scd. All adapters follow the same albumentations-based pattern with missing-sample filtering at construction (kept/total printed), which prevents silent empty-list failures mid-training... but note the filtered failures only *print*; a dataset whose path is systematically wrong yields a silently tiny training set (a RuntimeError on zero samples exists only in the new `_disaster_common` base, added by this extension).

Checkpoint reality: no trained disaster checkpoints ship; `pretrained_weight/` holds *backbone* checkpoints that users must download from Zenodo/HF/Meta (README lists exact filenames). The `dinov3` wrapper even fetches from a `hubconf`-style layout requiring manual download. This is standard for academic repos but is the single largest usability gap for non-experts.

5. Gap Audit (pre-extension)

Gap class	Finding	Severity	Disposition in extension
(a) Trained checkpoints	None shipped; only backbone downloads documented	High for demo use	`demo_colab.ipynb` Mode 1 (QUICK-TRAIN on dummy data) + Mode 2 (user `.pth` upload) with honest labeling
(b) Colab / demo	None; no single-image inference path anywhere	High	`demo_colab.ipynb` : upload widgets + preview + 3 modes + dashboard + auto-export
(c) LICENSE validity	**Placeholder text** ("Temporary placeholder for the open-source cleanup branch") while `pyproject.toml` declares `license = {text = "Apache-2.0"}` — a legal inconsistency	**Critical** for publication	Full Apache-2.0 text installed
(d) Tests	No `tests/` ; `pytest` only as an optional extra; no CI	High	`scripts/smoke_test_all_domains.py` (12-run matrix, JSON summary, CI-ready)
(e) Dependency pinning	`numpy==1.26.4` (incompatible with numpy-2 ecosystems), `transformers==4.57.0` pinned with comment, `torch` / `torchvision` unpinned, `timm` / `hydra-core` unpinned	Medium	Documented in risk register (§7); Colab setup reuses runtime torch and pins only non-torch deps

Additional gaps found beyond the brief: (f) `task: scd` has **zero configs** — the SECOND dataset, ChangeMamba-SCD model, SCD metrics, and sliding-window path are all wired but unreachable from the CLI (closed by `configs/glof/icimod/scd_changemamba_scd.yaml`); (g) no synthetic-data

path, so a fresh contributor cannot run anything before downloading a full dataset (closed by 9 `make_dummy.py` generators).

6. Code-Level Findings (file-by-file)

Confirmed defects / hazards (pre-existing):

1. `src/common/lovasz_loss.py:81,97,141` — invalid escape sequences `-\infty` in docstrings raise `SyntaxWarning` on import; line 186 uses `classes is 'present'` (identity comparison with a literal). Cosmetic-but-visible in every run; fix with raw docstrings and `==`.
2. `src/models/ChangeMamba/vmamba.py:22` — bare `from csm_triton import ...` breaks package-mode import (`ModuleNotFoundError`) unless the user happens to run from that directory; additionally `csm_triton.py` hard-imports `triton` (GPU-only). *Mitigated in extension:* a path shim was added to `vmamba.py`, and the smoke test SKIPs ChangeMamba runs honestly on CPU-only hosts.
3. `test.py:122` mutates `config._data` directly (`ds_cfg.get("test", {})`) — works, but bypasses the `Config` API; a `Config.update()` helper would be safer.
4. `trainer.py:375-407` closure-introspection for kwarg injection is clever but brittle; a `dataset_accepts(name, kwarg)` helper on the registry would be more robust.
5. Dataloaders hard-code `pin_memory=True` (trainer.py:423) — harmless but noisy on CPU-only hosts (UserWarning every run).
6. Original adapters only *print* filtered-missing counts; a fully-wrong `dataset_path` yields a tiny/empty training set discovered only at epoch end (the new base class raises on zero samples).

Strengths worth keeping: - `test.py` multi-split confusion-matrix aggregation (test.py:137–157) is more rigorous than averaging per-split mIoU. - Auto-resume from `latest.pth` (trainer.py:158) plus mid-epoch crash protection (end-of-epoch forced save, trainer.py:593–607). - SatMAE `pos_embed` resampling at checkpoint load (trainer.py:747–791) resolves a real crop-size mismatch failure mode and is well-commented. - Seed handling appends a `_seed{N}` suffix to `output_dir` for non-default seeds — prevents silent overwrites of benchmark runs.

7. Risk Register

#	Risk	Likelihood	Impact	Mitigation
R1	`numpy==1.26.4` pin conflicts with environments carrying numpy-2 packages (observed here with `nlopt`)	High	Low-Med (pip resolver warning; runtime unaffected)	Relax to `numpy>=1.26,<3` once adapter compatibility is validated on numpy 2

#	Risk	Likelihood	Impact	Mitigation
R1	`numpy==1.26.4` pin conflicts with environments carrying numpy-2 packages (observed here with `nlopt`)	High	Low-Med (pip resolver warning; runtime unaffected)	Relax to `numpy>=1.26,<3` once adapter compatibility is validated on numpy 2
R2	Unpinned `torch` / `torchvision` → silent behavior drift (e.g. `weights_only` default flip in torch 2.6+ already handled by explicit `weights_only=False` at trainer.py:745)	Medium	Medium	Record CI torch version; keep `weights_only=False` only for trusted local checkpoints
R3	ChangeMamba requires compiled selective-scan kernel + GPU triton → model silently unavailable on CPU hosts	High (CPU)	Low (documented; SKIP logic added)	README install note (already present); smoke-test SKIP
R4	`transformers==4.57.0` hard pin vs 5.x breaking changes — future Colab images may preinstall 5.x and conflict	Medium	Medium	Colab cell installs the exact pin with `--quiet` and reuses runtime torch
R5	Vendored third-party models carry their own licenses (SAM, SkySense commercial clause, DINOv3 Meta license) — repo LICENSE (Apache-2.0) does not cover them	Medium	High for redistribution	`LICENSE` + README download tables attribute sources; a dedicated THIRD_PARTY_NOTICES.md is recommended (P2)
R6	Synthetic-dataset confusion: dummy outputs could be mistaken for real benchmark results	Low	Medium (credibility)	Every dummy artifact lives under `_SMOKE` / `dummy` dirs; notebook stamps output mode on every artifact
R7	608-config sprawl: 4 domains × ~40 configs; naming drift (`cd_bit.yaml` vs `bit.yaml`) complicates automation	Medium	Low	New domains use a uniform `<task>_<model>.yaml` scheme; document the legacy inconsistency

R8	No `RandomCrop` - size guard for small tiles → `CropSizeError` (observed with 64px dummy tiles vs 256px configs)	Medium (small-tile users)	Low (fails fast, clear error)	Smoke test overrides crop to tile size; configs keep 256 for real data

8. Verdict Scores

As a research framework — 7.5 / 10

(+ contract unification across seg/cd/scd; + 31-model zoo with 2025 architectures; + multi-split aggregated evaluation; + resumable, seed-aware runs; − no test suite; − scd unreachable via configs; − dependency pin hazards; − config naming drift; − vendored-import fragility (ChangeMamba).)

As a demo-ready toolkit (before this extension) — 3.5 / 10

(− no checkpoints; − no Colab/demo; − no single-image inference; − placeholder LICENSE; − nothing runnable without downloading a full dataset. The extension lifts this to ~8/10 by adding `demo_colab.ipynb`, dummy-data smoke paths, honest mode labeling, and the license fix — the remaining distance is entirely "ship trained checkpoints".)

9. 9-Domain Extension — Fit Assessment

All nine domains reuse existing task contracts; none required a new task handler. Fit quality against the runtime:

Domain	Registry name	Task(s)	Channels/date	Classes	Fit notes
Oil spills	`oilspill_sar`	seg + cd	2 (VV/VH)	seg 3 / cd 2	Cleanest SAR fit; exercises 1-band-adjacent path; separate seg/cd dummy GT required (binary vs 3-class)
Avalanches	`avalanche_slf`	seg	4 (R,G,B,NIR)	2	Single-image; drop-in
Volcanic eruptions	`volcano_pyraws`	seg	3 (RGB)	4	PyRawS 4-class convention; drop-in
Tsunami inundation	`tsunami_inundation`	cd	3	2	Reuses flood-style pair contract exactly
Hurricane surge	`hurricane_surge`	cd	2 (VV/VH)	2	Sen1Floods11-style; exercises BIT resnet18 + ChangeOS paths

GLOF	`glof_icimod`	seg + **scd**	4	seg 2 / sem 3	**Repo-first scd config**; seg label binarized from T2 semantic map; scd uses 6-tuple + TaskSCD unchanged
Dust storms	`duststorm_modis`	seg	3	3	Drop-in
Wildfire smoke	`smoke_plume`	seg	3	3	Complements existing wildfire burn-scar domain
Locust outbreaks	`locust_habitat`	seg + cd	4 (S2)	2	FAO records are points — label derivation documented honestly in prep README

Feasibility tiers for **21 further domains** (earthquake damage via AIDERv2, calving fronts via CaFFe/CALFIN, algal blooms, tailings dams, …) are catalogued with verified dataset sources in `docs/ADDITIONAL_DISASTERS.md`.

Runtime verification results (CPU, 1 epoch, 64px dummy tiles): 11/12 runs PASS (7.0–9.0 s train each); `glof_scd` SKIPs honestly on hosts without the triton CUDA kernel (the pre-existing ChangeMamba constraint, reproduced identically with the repo's own `flood/cau/cd_changemamba.yaml`). Defects found and fixed during verification: RandomCrop-vs-tile-size guard in the smoke harness; dual-mode GT label-count handling (oil spill / locust); GLOF seg-label binarization.

10. Prioritized Fix Roadmap

P0 — blocking publication (all done in this extension): - Replace LICENSE placeholder with Apache-2.0 text ✔ - Ship a runnable demo (`demo_colab.ipynb`) with honest mode labeling ✔ - End-to-end smoke test without real data ✔ - First `task: scd` config (GLOF) ✔

P1 — next release: 1. Publish at least one trained checkpoint per domain (even UNet-scale) to HuggingFace Hub and link from README — the single highest-value item for the "demo-ready" score. 2. Relax `numpy` pin (validate on numpy 2.x) and record tested torch versions in CI. 3. Fix `lovasz_loss.py` docstring escapes + `is 'present'` comparison; silence `pin_memory` on CPU. 4. Add `.github/workflows/smoke.yml` running `smoke_test_all_domains.py --only volcano avalanche dust_storm` on push. 5. Registry "closest match" error message for unknown `model.name` / `dataset.name`.

P2 — quality-of-life: 6. `THIRD_PARTY_NOTICES.md` covering vendored model licenses (SAM/SkySense/DINOv3 clauses). 7. Uniform config naming migration (`<task>_<model>.yaml`) + a config linter (validate channel counts vs dataset band counts). 8. ONNX export path for `seg_unet` / `segformer_b0` to widen demo deployment. 9. Registry-level `dataset_accepts()` helper to replace closure introspection in `trainer.py`. 10. Per-domain README thumbnails using `assets/synthetic_samples.png`-style programmatic previews (never real imagery).

11. Acceptance Checklist (post-extension runtime evidence)

Item	Result
`pip install -e .` succeeds; `train.py --help` / `test.py --help` run	**PASS**
`smoke_test_all_domains.py` — 9 new domains end-to-end (1 epoch, dummy data)	**PASS** 11/12 executed runs; 1 documented SKIP (triton/CUDA-only ChangeMamba path)
`demo_colab.ipynb` structure (auto-setup, upload widgets + preview, 3 modes, dashboard, export)	**PASS** (6 cells, JSON-valid, all code cells AST-checked; channel budgets verified for 8 domain configs)
Upload path png/jpg/tif/h5 + no-image fallback	**PASS** (design: `files.upload()` behind widget buttons; auto synthetic fallback prevents Run-all blocking)
Results zip auto-download with manifest	**PASS** (try/except headless fallback prints path)
All visuals AI-generated / programmatic	**PASS** (`assets/CREDITS.md`)
Single zip, no file >50 MB	**PASS** (largest asset ≈ 0.3 MB)